Virtual Memory Page Replacement Visualizer

Operating Systems Project

Course: Operating Systems

Registration No.	Team Member
2520030431	Bolla Gopika Sai Sri Varshini
2520030436	Gottipati Eesha Yuktha
2520030556	Vanam Hamsini

Introduction

- Virtual memory: lets a program use more memory than the RAM available, by keeping only some of its pages in memory
- Page: a fixed-size block of a program's memory
- Page frame: a fixed-size slot in RAM that holds exactly one page
- Page replacement: when all frames are full and a new page is needed, one page must be removed

Small example

- RAM has 3 frames. Frames hold pages 1, 2, 3 (all full)
- The program now asks for page 4, so one of 1, 2 or 3 must be replaced
- The replacement algorithm decides which one

Problem Statement and Objectives

Problem

- Memory frames are limited
- Programs request many different pages
- When frames are full, one page must be replaced
- A poor choice causes more page faults and slows the program

Objectives

1. Simulate page replacement algorithms
2. Compare FIFO, LRU and Optimal
3. Count page hits and page faults
4. Calculate hit rates and fault rates
5. Generate a report of the results

Page Hits and Page Faults

- Page hit: the requested page is already in memory
- Page fault: the requested page is not in memory and must be loaded
- If all frames are occupied, a page must be replaced

Example: 3 frames, reference sequence 1, 2, 3, 1, 4 (FIFO replacement shown)

Request	**1**	**2**	**3**	**1**	**4**
Frame 1	1	1	1	1	4
Frame 2	-	2	2	2	2
Frame 3	-	-	3	3	3
Result	**Fault**	**Fault**	**Fault**	**Hit**	**Fault**

Requests 1, 2, 3: frames are empty, so these are faults. Second request for 1: hit. Request 4: fault and the frames are full, so page 1 is replaced.

FIFO Page Replacement Algorithm

- Replaces the page that entered memory first
- Follows the order in which pages were loaded (like a queue)
- A page hit does not change the replacement order

Example: 3 frames, sequence 7 0 1 2 0 3 0 4 2 3 0 3

Request	**7**	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0
Frame 2	-	0	0	0	0	3	3	3	2	2	2	2
Frame 3	-	-	1	1	1	1	0	0	0	3	3	3
Result	**F**	**F**	**F**	**F**	**H**	**F**	**F**	**F**	**F**	**F**	**F**	**H**

F = page fault, H = page hit. Result: 10 faults, 2 hits (out of 12 requests). When 2 arrives, 7 is oldest, so 7 is replaced.

LRU Page Replacement Algorithm

- Replaces the page that has not been used for the longest time
- Looks at recent page usage, not loading order
- Every access (hit or fault) updates the page's recent-use information

Example: 3 frames, sequence 7 0 1 2 0 3 0 4 2 3 0 3

Request	**7**	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**
Frame 1	7	7	7	2	2	2	2	4	4	4	0	0
Frame 2	-	0	0	0	0	0	0	0	0	3	3	3
Frame 3	-	-	1	1	1	3	3	3	2	2	2	2
Result	**F**	**F**	**F**	**F**	**H**	**F**	**H**	**F**	**F**	**F**	**F**	**H**

Result: 9 faults, 3 hits. When 3 arrives, page 1 was used longest ago, so 1 is replaced (page 0 was just used).

Optimal Page Replacement Algorithm

- Replaces the page whose next use is farthest in the future
- A page that is never used again is the best candidate to replace
- Gives the minimum page faults, so it is a theoretical benchmark
- Needs future knowledge of page references, so it is not practical in a real OS

Example: 3 frames, sequence 7 0 1 2 0 3 0 4 2 3 0 3

Request	**7**	**0**	**1**	**2**	**0**	**3**	**0**	**4**	**2**	**3**	**0**	**3**
Frame 1	7	7	7	2	2	2	2	2	2	2	0	0
Frame 2	-	0	0	0	0	0	0	4	4	4	4	4
Frame 3	-	-	1	1	1	3	3	3	3	3	3	3
Result	**F**	**F**	**F**	**F**	**H**	**F**	**H**	**F**	**H**	**H**	**F**	**H**

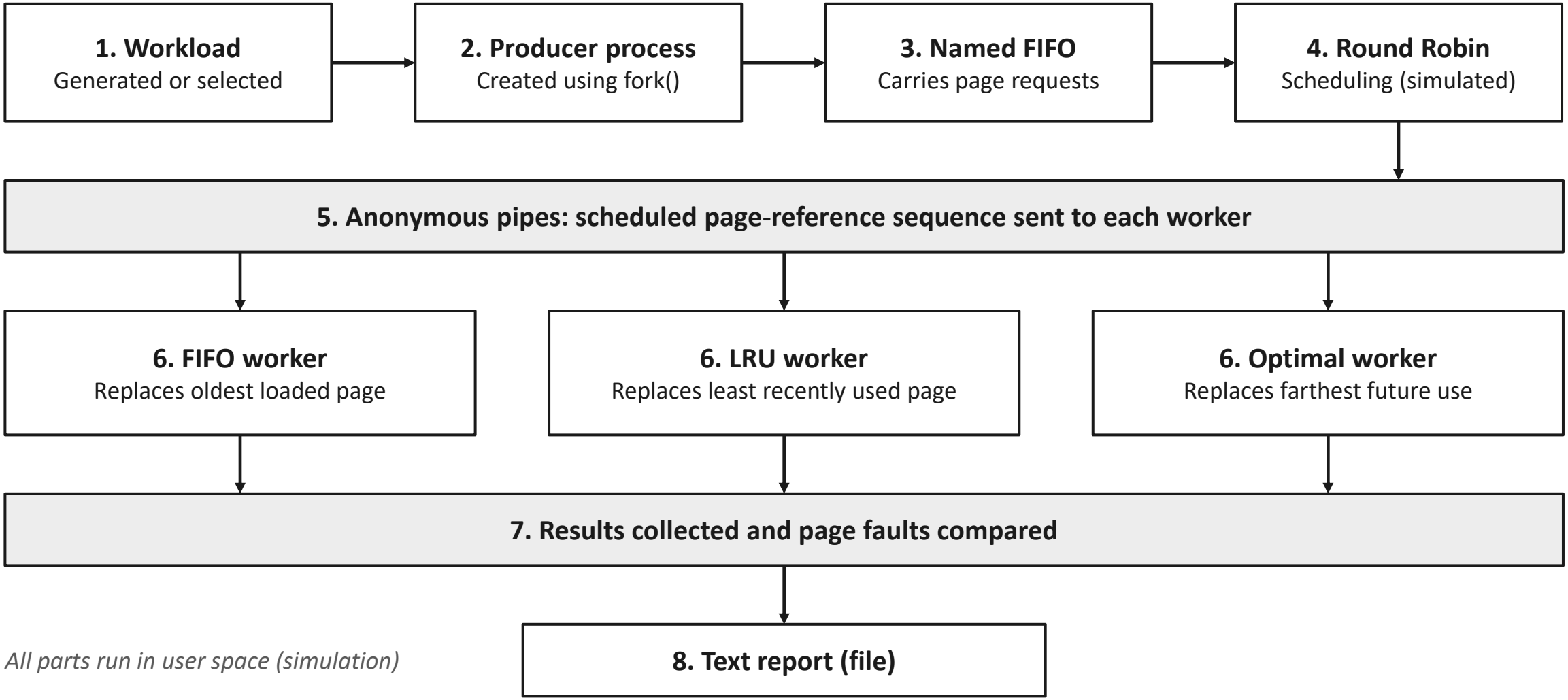
Result: 7 faults, 5 hits. When 4 arrives, page 0 is needed farthest in the future, so 0 is replaced.

System Design and Working

1. Generate or select a page-reference workload
2. Create a producer process using fork()
3. Transfer page requests through a named FIFO
4. Simulate Round Robin scheduling
5. Send the scheduled reference sequence to three worker processes
6. Run FIFO, LRU and Optimal independently
7. Collect results and compare page faults
8. Generate a text report

Flow: Workload > Producer (fork) > Named FIFO > Round Robin > 3 Workers (FIFO, LRU, Optimal) > Compare > Report

System Architecture Diagram



All parts run in user space (simulation)

Technologies and Implementation

Technology / Concept	How it is used
C programming language	Main implementation language
fork()	Linux process creation (producer and workers)
waitpid()	Waits for child processes to complete
Named FIFO	Inter-process communication: page requests
Anonymous pipes	Communication with the algorithm workers
Round Robin scheduling	Simulated to order the page requests
Arrays / data structures	Track pages and frames
File handling	Generates the report file

Note: Round Robin scheduling and virtual memory are simulated in user space. The project does not modify the real Linux kernel scheduler or page tables.

Results and Algorithm Comparison

Algorithm	Replacement Rule	Page Hits	Page Faults	Fault Rate
FIFO	Oldest loaded page	____	____	____
LRU	Least recently used page	____	____	____
Optimal	Farthest future use	____	____	____

- Placeholders: fill in from an actual program run (not invented numbers)
- Fault rate = (page faults / total references) x 100
- All three use the same scheduled page-reference sequence and the same number of frames
- Illustration only (slides 5 to 7, 12 requests, 3 frames): FIFO 10 faults, LRU 9, Optimal 7

Advantages, Limitations and Future Scope

Advantages (implemented)

- Makes page replacement easy to understand
- Compares 3 algorithms under identical conditions
- Tracks hits, faults and replacement details
- Supports predefined and custom workloads
- Generates a report of the results

Limitations

- Optimal needs knowledge of future references
- Simulates virtual memory; does not change real memory management
- Results depend on the workload and frame count

Future Scope (proposed, not implemented)

- Interactive graphical user interface
- More page replacement algorithms
- Larger workloads and more detailed performance analysis

Conclusion

- Page replacement decides which page to remove when memory frames are full
- FIFO uses loading order, LRU uses recent usage, Optimal uses future use
- Page faults help us evaluate how well each algorithm performs
- The project demonstrates operating-system concepts (processes, pipes, FIFO, scheduling) through simulation

Thank You – Questions?